

RTL Engineering Shadow

RTL Engineering Shadow - Engineering Design Specification

Sayyam Palrecha

July 22, 2026

Contents

1	Engineering Memory Manifesto	3
1.1	Purpose	3
1.2	The Problem	3
1.3	Our Beliefs	4
1.4	Design Principles	4
1.5	Vision	4
1.6	Success	5
2	Vision	5
3	Problem Statement	5
4	Primary User Persona	5
5	Current Workflow	5
6	Engineering Shadow Behavioral Specification	5
6.1	Purpose	5
6.2	Behavioral Philosophy	6
6.3	Behavioral Principles	6
6.4	Behavioral Summary	8
7	Product Hypothesis	8
8	MVP	8
9	Open Questions	9

10 Engineering Shadow Reference Model (ESRM)	10
10.1 Overview	10
10.2 Design Philosophy	10
10.3 Architecture Layers	11
10.4 Continuous Learning Loop	11
10.5 Architectural Significance	11
11 Project Understanding Model (PUM) Specification	12
11.1 Document Information	12
11.2 Purpose	12
11.3 Motivation	12
11.4 Definition	12
11.5 Design Philosophy	13
11.6 Core Responsibilities	13
11.7 Understanding Dimensions	13
11.8 Expected Capabilities	14
11.9 Interaction within the ESRM	14
11.10 Architectural Boundaries	15
11.11 Future Architectural Work	15
11.12 Summary	15

1 Engineering Memory Manifesto

1.1 Purpose

This project exists to address a fundamental limitation in modern engineering workflows: the inability to preserve and communicate engineering understanding as projects evolve.

Engineering projects are not defined solely by their source code, documentation, or design artifacts. They are driven by a continuous sequence of decisions, assumptions, experiments, trade-offs, and evolving understanding. Most of this knowledge exists only in the engineer's mind and gradually disappears over time.

The objective of this project is to preserve this engineering understanding in a structured, continuously evolving form.

1.2 The Problem

Modern engineering tools successfully manage artifacts.

Examples include:

- Source code
- Design files
- Version history
- Documentation
- Simulation results
- Reports

However, these tools rarely preserve the reasoning that produced those artifacts.

Questions such as the following often remain unanswered:

- Why was a particular architecture selected?
- Which alternatives were considered?
- What constraints influenced the decision?
- Why was an existing solution rejected?
- How did the design evolve over time?

As projects grow, this missing context becomes increasingly difficult to reconstruct.

1.3 Our Beliefs

We believe that:

1. Engineering knowledge extends beyond source code.
2. Design decisions are first-class engineering artifacts.
3. Engineering context continuously evolves throughout the project lifecycle.
4. Preserving reasoning is as important as preserving implementation.
5. Documentation should describe understanding rather than merely recording activities.
6. Engineering knowledge should remain accessible without relying on human memory.
7. Knowledge captured once should be reusable throughout future projects.

1.4 Design Principles

Every future design decision within this project should follow the principles below.

1. Minimize interruption to the engineer's workflow.
2. Preserve context rather than simply recording events.
3. Prefer understanding over observation.
4. Treat engineering decisions as persistent project knowledge.
5. Allow engineering knowledge to evolve as designs evolve.
6. Generate documentation from understanding rather than generating understanding from documentation.
7. Maintain transparency regarding captured knowledge and generated outputs.
8. Ensure that every generated artifact can be traced back to supporting engineering context.

1.5 Vision

We envision engineering environments where project knowledge no longer disappears as engineers move between tasks, meetings, iterations, or team members.

Instead, engineering understanding continuously evolves alongside the project itself, enabling efficient collaboration, onboarding, design reviews, and long-term knowledge preservation.

1.6 Success

This project will be considered successful if it enables engineers to communicate project context, design evolution, and engineering reasoning with significantly less effort than is required using current engineering workflows.

Success is measured not by the amount of generated documentation, but by the quality, continuity, and accessibility of preserved engineering understanding.

2 Vision

Status: Draft

TODO:

- Long-term vision
- Mission
- North Star

3 Problem Statement

Status: Empty

Describe the engineering problem, why it exists, who experiences it, and why current tools fail.

4 Primary User Persona

- RTL Design Engineer
- PhD Student
- Hardware Design Team

5 Current Workflow

Capture the user's real RTL workflow before proposing solutions.

6 Engineering Shadow Behavioral Specification

6.1 Purpose

This document defines the expected behavior of Engineering Shadow independent of implementation details.

Its purpose is to ensure that every implementation of Engineering Shadow exhibits consistent behavior regardless of the underlying technologies, programming languages, AI models, or deployment environments.

The principles defined in this document represent permanent behavioral characteristics of the product and shall guide all future product, architecture, and engineering decisions.

6.2 Behavioral Philosophy

Engineering Shadow exists to reduce the cognitive overhead associated with long-term RTL engineering projects.

The Shadow does not replace the engineer.

The Shadow continuously develops and maintains an evolving understanding of the project so that engineering assistance becomes increasingly context-aware as the project progresses.

The Shadow's objective is not to maximize automation.

Its objective is to maximize engineering understanding while minimizing interruption.

6.3 Behavioral Principles

Principle 1: Minimize Cognitive Load

Statement

Every interaction initiated by the Shadow shall reduce, or at minimum not increase, the engineer's cognitive load.

Rationale

Engineers already operate under significant cognitive demand while solving complex design problems. Excessive prompts, notifications, or unnecessary interactions reduce productivity and discourage long-term use.

Example

The Shadow remains silent while the engineer edits source code.

The Shadow asks for clarification only after detecting a significant engineering event.

Principle 2: Understanding Before Assistance

Statement

The Shadow shall establish sufficient project understanding before providing project-specific recommendations.

Rationale

Recommendations generated without sufficient context are often generic, inconsistent with project constraints, or technically incorrect.

Project understanding shall always precede project-specific assistance.

Example

Before recommending an RTL implementation strategy, the Shadow understands:

- project objectives,
- architectural constraints,
- implementation history,
- current engineering stage.

Principle 3: Engineer Intent is Authoritative

Statement

The engineer remains the authoritative source of engineering intent.

The Shadow may infer intent when confidence is sufficiently high but shall request clarification whenever uncertainty could significantly affect project understanding.

Rationale

Engineering decisions frequently involve trade-offs that cannot always be inferred reliably from observed activities alone.

The engineer's explanation takes precedence over automated inference.

Example

The Shadow detects a substantial architectural modification.

Rather than assuming the reason, it requests a brief explanation before updating project understanding.

Principle 4: Preserve Engineering Reasoning

Statement

Engineering reasoning shall be treated as a first-class project artifact.

Rationale

Design decisions derive their value from the reasoning that produced them.

Without preserved reasoning, future engineers must reconstruct context through repeated discussions or investigation.

Example

When an architecture changes, the Shadow records not only the new architecture but also the motivation, alternatives considered, and engineering constraints that influenced the decision.

Principle 5: Observe Continuously, Interrupt Selectively

Statement

Observation shall be continuous.

Interruption shall be selective.

Rationale

Most engineering activities provide useful context but do not require immediate interaction.

Interruptions shall occur only when clarification is necessary to preserve accurate project understanding.

Example

Reading research papers requires no interruption.

Replacing a core architectural component may require clarification.

Principle 6: Maintain an Evolving Project Understanding

Statement

The Shadow shall continuously update its understanding as the project evolves.

Project understanding is never considered complete.

Rationale

Engineering projects evolve through experimentation, validation, redesign, and changing priorities.

The Shadow must evolve alongside the project rather than relying on static documentation.

Example

Changes to project objectives, architecture, constraints, verification results, or guide feed-back update the Shadow's understanding.

Principle 7: Communicate with Transparency**Statement**

The Shadow shall clearly distinguish between observed facts, inferred conclusions, and engineering recommendations.

Rationale

Engineers must understand the origin of information before making design decisions.

Transparency increases trust and allows incorrect assumptions to be corrected quickly.

Example

The Shadow communicates whether a conclusion originated from:

- direct engineer input,
- observed engineering activities,
- project artifacts,
- automated inference.

6.4 Behavioral Summary

Engineering Shadow shall behave as a collaborative engineering partner that continuously develops project understanding while minimizing interruption to the engineer's workflow.

Its primary responsibility is to maintain sufficient project understanding to provide context-aware engineering assistance throughout the RTL engineering lifecycle.

The Shadow shall prioritize understanding over automation, reasoning over activity logging, and engineering context over isolated interactions.

7 Product Hypothesis

State the core hypothesis this product aims to validate.

8 MVP

Define only the minimum functionality required for validation.

9 Open Questions

Maintain unanswered product and technical questions here.

10 Engineering Shadow Reference Model (ESRM)

10.1 Overview

The Engineering Shadow Reference Model (ESRM) defines the highest-level conceptual architecture of the Engineering Shadow platform.

Rather than describing implementation details, the ESRM explains how engineering information flows through the system and gradually evolves into meaningful engineering understanding.

The reference model serves as the conceptual foundation upon which all future architectural, implementation, and product decisions are based.

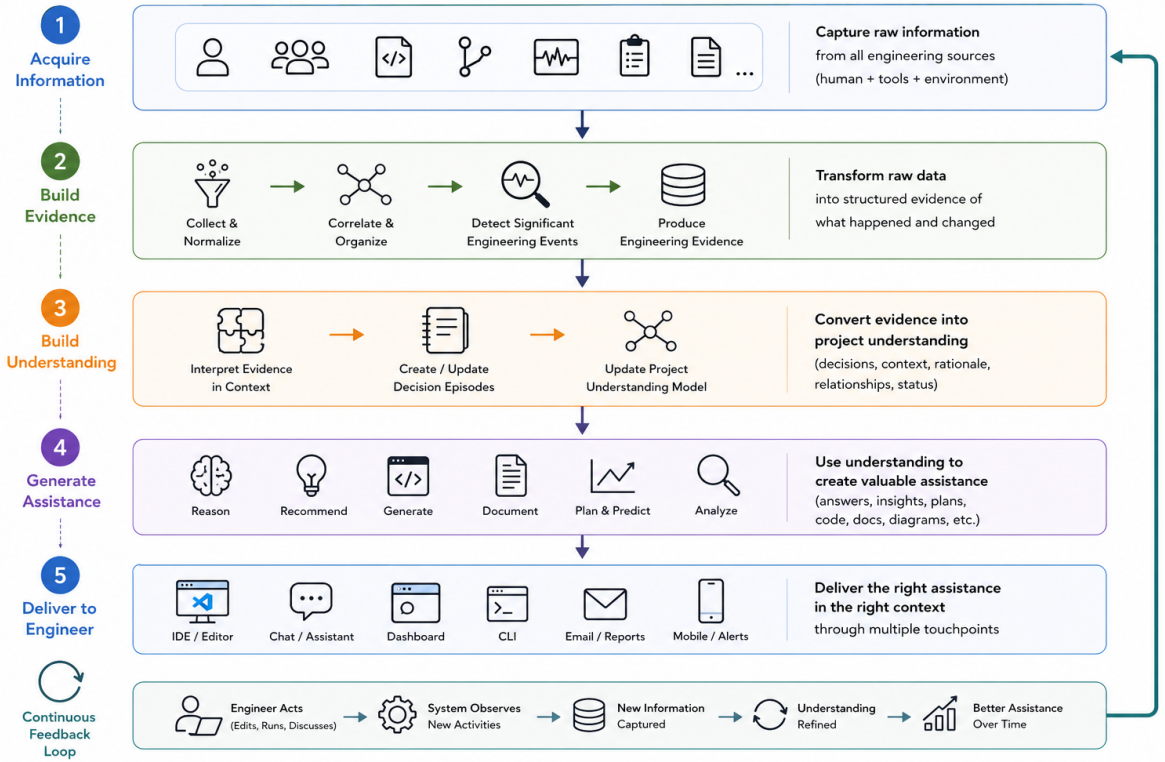


Figure 1: Engineering Shadow Reference Model Version 1

10.2 Design Philosophy

The architecture follows a simple principle:

Information → Evidence → Understanding → Assistance

Rather than generating assistance directly from raw engineering activities, the system first constructs an evolving understanding of the project.

Engineering assistance is therefore generated from accumulated project understanding instead of isolated prompts or individual engineering events.

10.3 Architecture Layers

The ESRM consists of four conceptual layers.

1. Information Acquisition

Collect engineering information from both explicit and implicit sources including engineers, engineering tools, source code, documentation and project artifacts.

2. Evidence Construction

Transform raw engineering information into structured engineering evidence by correlating observations, detecting engineering events and filtering irrelevant activities.

3. Understanding Engine

Interpret engineering evidence to construct Decision Episodes and continuously maintain the Project Understanding Model (PUM).

4. Assistance Engine

Leverage the current Project Understanding Model to generate context-aware engineering assistance including recommendations, documentation, reasoning, planning and engineering insights.

10.4 Continuous Learning Loop

Engineering Shadow operates as a continuously evolving system.

Every engineer interaction, engineering activity, design decision and project artifact contributes new information that refines the Project Understanding Model.

As project understanding improves, the quality of engineering assistance also improves.

10.5 Architectural Significance

The ESRM intentionally separates project understanding from engineering assistance.

This distinction enables multiple intelligent capabilities to share a common understanding of the project while remaining independent of individual user interfaces, AI models or implementation technologies.

Consequently, the Project Understanding Model becomes the central knowledge asset of the Engineering Shadow platform.

11 Project Understanding Model (PUM) Specification

11.1 Document Information

Document	Project Understanding Model (PUM) Specification
Version	1.0
Status	Draft
Last Updated	July 2026
Related Architecture	Engineering Shadow Reference Model v1

11.2 Purpose

The Project Understanding Model (PUM) is the central knowledge component of the Engineering Shadow platform.

Its purpose is to continuously maintain an evolving understanding of an engineering project by integrating engineering evidence collected throughout the project lifecycle.

Rather than acting as a storage system for engineering artifacts, the PUM represents the Shadow’s current understanding of the project, enabling context-aware reasoning and intelligent engineering assistance.

Within the Engineering Shadow Reference Model (ESRM), the PUM serves as the primary output of the Understanding Engine and the primary input to the Assistance Engine.

11.3 Motivation

Modern engineering projects generate large amounts of information including source code, documentation, simulation results, design reviews, research papers, engineering discussions and tool outputs.

Although these artifacts contain valuable information, they rarely capture the reasoning, relationships and project context required to understand how a project evolved.

Consequently, engineers repeatedly reconstruct project context before making decisions, reviewing designs or continuing previous work.

The Project Understanding Model addresses this problem by continuously transforming engineering evidence into an evolving project understanding that can be reused throughout the project lifecycle.

11.4 Definition

The Project Understanding Model (PUM) is defined as:

A continuously evolving representation of both the structural knowledge and the current situational state of an engineering project, enabling Engineering Shadow to reason about what the project is, how it evolved and what should happen next.

The PUM does not attempt to replicate the complete cognitive state of an engineer.

Instead, it maintains sufficient project understanding to provide meaningful, context-aware engineering assistance.

11.5 Design Philosophy

The design of the PUM follows four fundamental principles.

1. Understanding over Storage

The objective of the PUM is to maintain engineering understanding rather than archive engineering information.

2. Relationships over Isolated Facts

Engineering understanding emerges from relationships between project concepts rather than independent pieces of information.

3. Continuous Evolution

Project understanding evolves continuously as new engineering evidence becomes available.

4. Reasoning before Assistance

Engineering assistance is generated from project understanding rather than directly from raw engineering information.

11.6 Core Responsibilities

The Project Understanding Model is responsible for:

- Maintaining an evolving understanding of the engineering project.
- Preserving engineering rationale throughout the project lifecycle.
- Capturing relationships between engineering concepts.
- Maintaining awareness of the project's current situation.
- Providing a consistent project context for intelligent engineering assistance.
- Supporting traceability between engineering evidence and project understanding.

11.7 Understanding Dimensions

The PUM maintains understanding across multiple complementary dimensions.

1. Project Context

Overall understanding of the project's objectives, scope and engineering intent.

2. Architectural Understanding

Understanding of the system architecture and relationships between engineering components.

3. Decision Understanding

Understanding of why engineering decisions were made and how they influenced the project.

4. Requirement Understanding

Understanding of the engineering requirements, constraints and design drivers.

5. Verification Understanding

Understanding of verification activities and confidence in engineering outcomes.

6. Progress Understanding

Understanding of the current state of the project, ongoing work and active engineering priorities.

7. Issue Understanding

Understanding of unresolved engineering problems, blockers and associated context.

11.8 Expected Capabilities

A mature Project Understanding Model should enable Engineering Shadow to answer questions such as:

- What is this project trying to achieve?
- What is the current engineering objective?
- Why was a particular design decision made?
- Which requirements influenced this architecture?
- How has the architecture evolved over time?
- What engineering issues remain unresolved?
- What evidence supports the current design?
- What should be considered before making the next engineering decision?

These questions define the expected behavior of the PUM rather than its internal implementation.

11.9 Interaction within the ESRM

Within the Engineering Shadow Reference Model:

- The Information Acquisition Layer provides raw engineering information.
- The Evidence Construction Layer transforms observations into engineering evidence.
- The Understanding Engine updates the Project Understanding Model using the available evidence.
- The Assistance Engine generates engineering assistance using the current Project Understanding Model.

Consequently, the PUM acts as the central knowledge interface between engineering understanding and engineering assistance.

11.10 Architectural Boundaries

The Project Understanding Model intentionally does not define:

- Internal data structures.
- Storage technologies.
- Database schemas.
- Knowledge graph implementations.
- AI models.
- Algorithms for evidence processing.
- User interfaces.

These concerns belong to subsequent architectural specifications.

11.11 Future Architectural Work

This specification establishes the conceptual role of the PUM.

Future documents will describe:

- Engineering Ontology
- Engineering Entities
- Engineering Relationships
- Decision Episode Integration
- Project Understanding Evolution
- Evidence Integration
- Internal PUM Architecture
- Reasoning Mechanisms

11.12 Summary

The Project Understanding Model is the central knowledge asset of Engineering Shadow.

Rather than functioning as a repository of engineering information, it maintains a continuously evolving understanding of the engineering project.

By separating project understanding from engineering assistance, the PUM provides a stable architectural foundation upon which future reasoning, automation and intelligent engineering capabilities can be built.